ECPS 216: IOT Systems & Software

Assignment5- CodeRED Report

Prepared by - Pranav Upadhyay (75859028/upadhyp1)

Demonstration Information

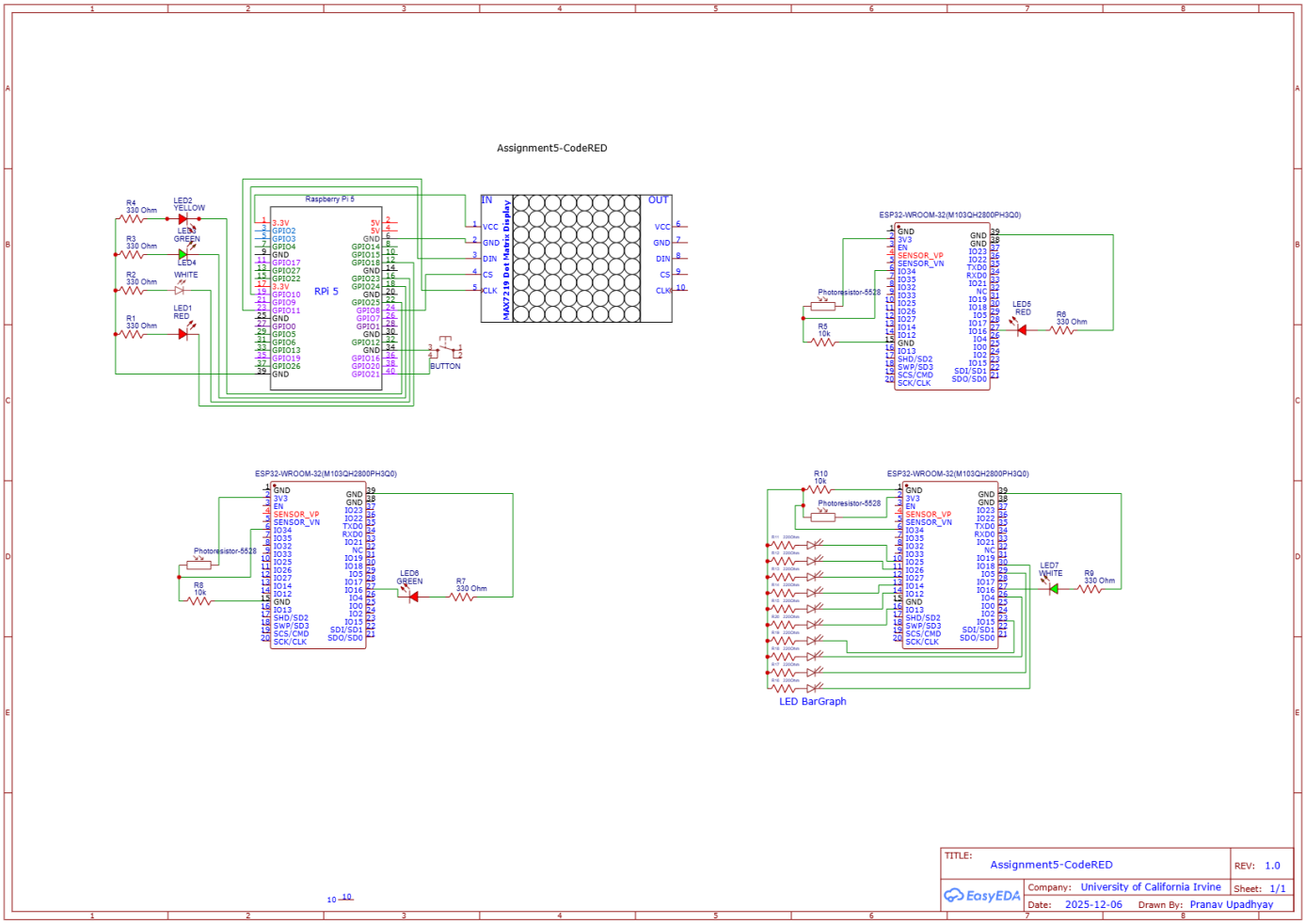
Video Link

https://drive.google.com/file/d/14y6Y1_WKGqxO-bY-Ug3RDmKU0RVARJx7/view?usp=sharing

Key Features Demonstrated:

- Dynamic master election among 3 ESP32 devices
- LED bar graphs on ESP32 showing light intensity
- MAX7219 LED Matrix displaying 30-second rolling trace
- Real-time web dashboard with interactive charts
- Reset button functionality with yellow LED indicator
- Automatic log file management with timestamped filenames
- Web-accessible historical log viewer

Overview - Schematic and Communication Protocol



Direction	Transport	Port	Message Type	Message Format	Fields	Example Payload	Description
ESP → RPi	UDP	5005	Photocell Data Packet	JSON string	master (string, ESP IP) value (int, ADC reading)	{"master":"192.168.1.40","value":1580}	ESP32 sends sensor reading + the ESP acting as "current master." RPi uses this to update LED indicators, matrix accumulation, and master duration tracking.
ESP → RPi	UDP	5005	Heartbeat / Presence	JSON string (same as above)	master, value	{"master":"192.168.1.40","value":1625}	Even if value doesn't change, ESP packets act as periodic heartbeat to show the device is alive and connected.
RPi → ESP (broadcast)	UDP	5005	Reset Command	Plain ASCII string	None	RESET	RPi sends this broadcast when the physical button is pressed. All ESPs must listen and trigger their internal reset or data-clear routine.
RPi → Browser	HTTP (GET)	80xx (Flask default)	Dashboard Page	HTML/JS	None	(HTML rendered at /)	Used for real-time human monitoring. Displays JSON polled from /api/current_data.
Browser → RPi	HTTP (GET)	80xx	Live Data Request	HTTP request	None	GET /api/current_data	Browser asks for latest swarm status (master, values, LED matrix data, durations).
Browser → RPi	HTTP (GET)	80xx	Request list of log files	HTTP request	None	GET /api/logs	Dashboard requests all saved swarm session logs.
Browser → RPi	HTTP (GET)	80xx	Download log file	HTTP request	Filename	GET /logs/swarm_log_2025...json	Allows user to download saved log files for analysis.
RPi → Browser	HTTP (JSON Response)	80xx	Log File Content	JSON	session_start, session_end, master_durations, data_points[], etc.	JSON object containing entire saved session	Used when user selects a log to view directly in dashboard.

Part1: RaspberryPi Setup

Raspberry Pi System Flowchart

```
                    ┌──────────────────┐
                    │      BOOT UP     │
                    │  OS + Python Start │
                    └──────────┬───────┘
                               ↓
                    ┌──────────────────┐
                    │   INITIALIZATION │
                    │ Setup GPIO, SPI, │
                    │ Flask, Threads   │
                    └──────────┬───────┘
                               ↓
          ┌──────────────────────────────────┐
          │              IDLE STATE          │
          │ Waiting for UDP packets & button input │
          └──────────┬──────────┬────────────┘
                     ↓          ↓
   ┌──────────────────────┐   ┌──────────────────────┐
   │ UDP PACKET RECEIVED  │   │   BUTTON PRESSED     │
   │ Parse JSON {master,  │   │ Save log, blink LED, │
   │ value}, update data  │   │ reset session, send  │
   │ structures           │   │ RESET to ESPs        │
   └──────────┬──────────┘   └──────────┬──────────┘
              ↓                          ↓
   ┌──────────────────────┐   ┌──────────────────────┐
   │ UPDATE LED MATRIX    │   │ NEW SESSION STARTED  │
   │ Average readings,    │   │ Clear buffers & timer │
   │ write to MAX7219     │   │                      │
   └──────────┬──────────┘   └──────────┬──────────┘
              └──────────────→ (Return to IDLE STATE)
```

Explanation:

State	Description
BOOT UP	The Raspberry Pi loads the OS and starts the Python script via user execution or systemd. No network communication happens yet.
INITIALIZATION	GPIO pins, SPI interface, MAX7219 LED matrix, and the Flask web server are set up. The Pi automatically connects to WiFi through Raspberry Pi OS networking. UDP listener and button monitor threads are launched.
IDLE STATE	The Pi waits for UDP packets on port 5005 from ESP32 devices. It simultaneously monitors the button pin for press events.
UDP PACKET RECEIVED	When the Pi receives JSON data {"master":"<ip>","value":<adc>}, it parses the message, logs the reading, updates the current master, and stores values for LED matrix averaging.
UPDATE LED MATRIX	Every 4 seconds, the Pi computes an average of collected readings and updates a column on the MAX7219 LED matrix to visualize light intensity trends.
BUTTON PRESSED	A button presses triggers session log file creation, a yellow LED blink notification, and a RESET UDP broadcast to all ESPs. All session buffers are cleared to start a new recording cycle.
NEW SESSION STARTED	The Raspberry Pi resets counters, timers, histories, and internal structures to begin a clean, fresh session without previous data.

Main functionality of RaspberryPi code

Category	Details
INPUT	• UDP packets from ESP32 devices (port 5005, JSON containing master and value) • Pushbutton signal on GPIO 21 • HTTP GET requests from the browser dashboard
PROCESS	• Parse JSON sensor values and validate incoming data • Track master durations and maintain rolling histories for visualization & logging • Accumulate sensor readings for LED matrix updates and compute 4-second averages • Build session logs (timestamps, sensor data, master transitions) • Handle button-press workflow: save logs, clear buffers, and broadcast RESET
OUTPUT	• Update GPIO LEDs to indicate active ESP devices and button actions • Update MAX7219 LED matrix via SPI to visualize sensor activity • Serve Flask dashboard & REST API endpoints (/api/current_data, /logs/...) • Save session logs as JSON files on disk • Broadcast "RESET" command to all ESP devices over UDP

Data Collection and log writing process:

```
┌──────────────────┐
│  UDP LISTENER LOOP │
└──────────┬─────────┘
           ↓
┌──────────────────────────┐
│ Receive UDP JSON packet  │
│ {master, value}          │
└──────────┬───────────────┘
           ↓
┌──────────────────────────┐
│ Validate + parse JSON    │
└──────────┬───────────────┘
           ↓
┌──────────────────────────┐
│ Store in memory:         │
│ - photocell_history      │
│ - current_log_data       │
│ - master_durations       │
└──────────┬───────────────┘
           ↓
┌──────────────────────────┐
│ Accumulate for LED matrix │
└──────────┬───────────────┘
           ↓ (every 4 seconds)
┌──────────────────────────┐
│ Compute average & write to │
│ LED matrix (MAX7219)     │
└──────────┬───────────────┘
           ↓
┌──────────────────────────┐
│  On button press → generate │
│  log file JSON on disk   │
└──────────────────────────┘
```

Data Structure:

Data Structure	Type	Contents / Fields	Purpose / Description
photocell_history	collections.deque	{ "timestamp": ISO8601, "value": int, "master": "ip" }	Rolling window of the most recent sensor readings for real-time display and LED matrix updates. Helps visualize short-term light changes.
current_log_data	list	Same structure as photocell_history but stores *entire session*	Full chronological record of all sensor readings received during a session. Used for generating session log files.
master_durations	defaultdict(float)	Key: "ip_address", Value: total seconds	Tracks how long each ESP32 device acted as the swarm "master." Updated dynamically each time the master changes.
matrix_data	collections.deque (max length 8)	Float values (averaged ADC readings)	Holds the last 8 averaged 4-second sensor values. Each entry maps to one column of the MAX7219 8×8 LED matrix.
matrix_accumulator	list	Raw values collected over each 4-second window	Temporary storage for values used to compute the average for each LED matrix update.
known_esps	dict	Maps ESP IP → assigned LED GPIO pin	Keeps track of which ESP device is assigned to which indicator LED on the Raspberry Pi.
current_master	string	IPv4 address of active master ESP	Indicates which ESP currently controls the swarm based on sensor values.
session_start_time	string (ISO8601)	Timestamp the session began	Used to label log files and track session duration.
LOG_DIR	string ("swarm_logs")	Directory path for saved logs	Central location where JSON log files are stored for download or later analysis.

Example output logfile:

File Edit Selection View Go Run Terminal Help

swarm_log_20251206_194614.json

E: > IOT > Assignment5-CodeRED > swarm_logs > swarm_log_20251206_194614.json > session_start


```
{
  "session_start": "2025-12-06T19:46:03.363332",
  "session_end": "2025-12-06T19:46:14.380651",
  "master_durations": {
    "172.20.10.5": 0.0
  },
  "data_points": [
    {
      "timestamp": "2025-12-06T19:46:05.301128",
      "master": "172.20.10.5",
      "value": 1069
    },
    {
      "timestamp": "2025-12-06T19:46:05.317048",
      "master": "172.20.10.5",
      "value": 1058
    },
    {
      "timestamp": "2025-12-06T19:46:05.317356",
      "master": "172.20.10.5",
      "value": 1062
    },
    {
      "timestamp": "2025-12-06T19:46:05.327503",
      "master": "172.20.10.5",
      "value": 1063
    },
    {
      "timestamp": "2025-12-06T19:46:05.338525",
      "master": "172.20.10.5",
      "value": 1059
    },
    {
      "timestamp": "2025-12-06T19:46:05.349146",
      "master": "172.20.10.5",
      "value": 1070
    },
```

Part 2 - Generative AI Usage

Parts generated using AI:

Part to create the webpage:

```
# ===== HTML Template =====
def create_html_template():
    """Create the dashboard HTML template"""
    template_dir = 'templates'
    if not os.path.exists(template_dir):
        os.makedirs(template_dir)

    html_content = '''<!DOCTYPE html>
<html>
<head>
    <title>ESP32 Swarm Monitor</title>
    <script
src="https://cdn.jsdelivr.net/npm/chart.js"></s
cript>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
            background-color: #f5f5f5;
        }
        .container {
            max-width: 1400px;
            margin: 0 auto;
        }
        h1 {
            color: #333;
            text-align: center;
        }
        .mode-selector {
            background: white;
            padding: 15px;
            border-radius: 8px;
            margin-bottom: 20px;
            box-shadow: 0 2px 4px
rgba(0,0,0,0.1);
            text-align: center;
        }
        .mode-btn {
            padding: 10px 30px;
            margin: 0 10px;
            border-radius: 4px;
            border: 2px solid #4CAF50;
            background-color: white;
            color: #4CAF50;
            cursor: pointer;
            font-size: 16px;
            font-weight: bold;
        }
        .mode-btn.active {
            background-color: #4CAF50;
            color: white;
        }
        .mode-btn:hover {
            background-color: #45a049;
            color: white;
        }
        .status {
            background: white;
            padding: 15px;
            border-radius: 8px;
            margin-bottom: 20px;
            box-shadow: 0 2px 4px
rgba(0,0,0,0.1);
        }
        .charts {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 20px;
            margin-bottom: 20px;
        }
        .chart-container {
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 4px
rgba(0,0,0,0.1);
        }
        .log-section {
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 4px
rgba(0,0,0,0.1);
        }
        .file-upload-section {
            background: white;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 2px 4px
rgba(0,0,0,0.1);
            margin-bottom: 20px;
        }
        select, button, input[type="file"] {
            padding: 8px 15px;
            margin: 5px;
            border-radius: 4px;
            border: 1px solid #ddd;
        }
        button {
            background-color: #4CAF50;
            color: white;
            cursor: pointer;
        }
        button:hover {
            background-color: #45a049;
        }
        .master-indicator {
            display: inline-block;
            padding: 5px 10px;
            background-color: #4CAF50;
            color: white;
            border-radius: 4px;
            font-weight: bold;
        }
        .info-box {
            background: #e3f2fd;
            padding: 10px;
            border-radius: 4px;
            margin-top: 10px;
            font-size: 14px;
        }
        .offline-indicator {
            background: #ff9800;
            color: white;
            padding: 10px;
            border-radius: 4px;
            text-align: center;
            font-weight: bold;
            margin-bottom: 15px;
        }
        .online-indicator {
            background: #4CAF50;
            color: white;
            padding: 10px;
            border-radius: 4px;
            text-align: center;
            font-weight: bold;
            margin-bottom: 15px;
        }
        #offlineMode, #liveMode {
            display: none;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>ESP32 Swarm Monitor
Dashboard</h1>

        <div class="mode-selector">
            <button class="mode-btn active"
id="liveModeBtn"
onclick="switchMode('live')">Live
Mode</button>
            <button class="mode-btn"
id="offlineModeBtn"
onclick="switchMode('offline')">Offline
Mode</button>
        </div>

        <!-- LIVE MODE -->
        <div id="liveMode">
            <div id="connectionStatus"
class="online-indicator">
                Connected - Receiving Live Data
            </div>

            <div class="status">
                <h3>Current Status</h3>
                <p><strong>Session
Started:</strong> <span id="sessionStart">-
</span></p>
                <p><strong>Current
Master:</strong> <span id="currentMaster"
class="master-indicator">None</span></p>
                <p><strong>Known ESPs:</strong>
<span id="knownEsps">-</span></p>
                <div class="info-box">
                    <strong>LED Matrix:</strong>
Displays photocell trace over last ~30 seconds
(8 columns × 4 seconds each)
                </div>
            </div>

            <div class="log-section">
                <h3>Download Current Session
Logs</h3>
                <select id="logSelect">
                    <option value="">Select a log
file...</option>
                </select>
                <button
onclick="downloadLog()">Download
Selected Log</button>
            </div>
        </div>

        <!-- OFFLINE MODE -->
        <div id="offlineMode">
            <div class="offline-indicator">
                Offline Mode - Viewing Saved Log
Data
            </div>
            <div class="file-upload-section">
```

```
            <h3>Load Log File</h3>
            <p><strong>Option 1:</strong>
Upload a log file from your computer</p>
            <input type="file" id="fileInput"
accept=".json" onchange="loadLocalFile()">

            <p style="margin-top:
20px;"><strong>Option 2:</strong> Load
from Raspberry Pi (if available)</p>
            <select id="offlineLogSelect">
                <option value="">Select a log file
from Pi...</option>
            </select>
            <button
onclick="loadPiLog()">Load from
Pi</button>
        </div>
        <div class="status" id="offlineStatus"
style="display: none;">
            <h3>Log File Information</h3>
            <p><strong>Session Start:</strong>
<span id="offlineSessionStart">-</span></p>
            <p><strong>Session End:</strong>
<span id="offlineSessionEnd">-</span></p>
            <p><strong>Total Data
Points:</strong> <span
id="offlineTotalPoints">-</span></p>
            <p><strong>Master ESPs:</strong>
<span id="offlineMasters">-</span></p>
        </div>
    </div>

    <!-- CHARTS (Shared between modes) --
>
    <div class="charts">
        <div class="chart-container">
            <h3>Photocell Data Over
Time</h3>
            <canvas
id="photocellChart"></canvas>
        </div>

        <div class="chart-container">
            <h3>Master Duration
(seconds)</h3>
            <canvas
id="masterChart"></canvas>
        </div>
    </div>
  </div>

  <script>
    let photocellChart, masterChart;
    let updateInterval;
    let currentMode = 'live';
    let isServerAvailable = true;
    let smoothedValues = []; // For smoothing
light sensor readings

    // Simple moving average function for
smoothing
    function movingAverage(values,
windowSize = 3) {
        if (values.length < windowSize) return
values;

        let smoothed = [];
        for (let i = 0; i < values.length; i++) {
            let start = Math.max(0, i -
Math.floor(windowSize / 2));
            let end = Math.min(values.length, i
+ Math.ceil(windowSize / 2));
            let sum = 0;
            let count = 0;

            for (let j = start; j < end; j++) {
                sum += values[j];
                count++;
            }
            smoothed.push(sum / count);
        }
        return smoothed;
    }

    // Initialize charts
    function initCharts() {
        const photocellCtx =
document.getElementById('photocellChart').g
etContext('2d');
        photocellChart = new
Chart(photocellCtx, {
            type: 'line',
            data: {
                labels: [],
                datasets: [{
                    label: 'Light Value',
                    data: [],
                    borderColor: 'rgb(75, 192,
192)',
                    backgroundColor: 'rgba(75,
192, 192, 0.2)',
                    tension: 0.4,  // Increased from
0.1 for smoother curves
                    borderWidth: 2,
                    pointRadius: 1,  // Smaller
points for cleaner look
                    pointHoverRadius: 4
                }]
            },
            options: {
                responsive: true,
                scales: {
                    y: {
                        beginAtZero: true,
                        max: 4095,
                        title: { display: true, text:
'Photocell Value' }
                    },
                    x: {
                        title: { display: true, text:
'Time' }
                    }
                },
                animation: {
                    duration: 750,  // Smooth
animation instead of instant
                    easing: 'easeInOutQuart'
                },
                transitions: {
                    active: {
                        animation: {
                            duration: 400
                        }
                    }
                }
            }
        });

        const masterCtx =
document.getElementById('masterChart').getC
ontext('2d');
        masterChart = new Chart(masterCtx, {
            type: 'bar',
            data: {
                labels: [],
                datasets: [{
                    label: 'Duration (seconds)',
                    data: [],
                    backgroundColor: [
                        'rgba(255, 99, 132, 0.6)',
                        'rgba(54, 162, 235, 0.6)',
                        'rgba(255, 206, 86, 0.6)'
                    ]
                }]
            },
            options: {
                responsive: true,
                scales: {
                    y: {
                        beginAtZero: true,
                        title: { display: true, text:
'Time as Master (s)' }
                    }
                },
                animation: {
                    duration: 500,
                    easing: 'easeInOutQuart'
                }
            }
        });
    }

    // Switch between live and offline mode
    function switchMode(mode) {
        currentMode = mode;

        if (mode === 'live') {

document.getElementById('liveMode').style.di
splay = 'block';

document.getElementById('offlineMode').styl
e.display = 'none';

document.getElementById('liveModeBtn').cla
ssList.add('active');

document.getElementById('offlineModeBtn').
classList.remove('active');

            // Clear charts and restart live
updates
            clearCharts();
            updateCurrentData();  // Immediate
update
            if (updateInterval)
clearInterval(updateInterval);
            updateInterval =
setInterval(updateCurrentData, 1000);  //
Changed from 500ms to 1000ms for smoother
updates
        } else {

document.getElementById('liveMode').style.di
splay = 'none';

document.getElementById('offlineMode').styl
e.display = 'block';

document.getElementById('liveModeBtn').cla
ssList.remove('active');

document.getElementById('offlineModeBtn').
classList.add('active');

            // Stop live updates
            if (updateInterval) {
                clearInterval(updateInterval);
                updateInterval = null;
            }

            // Clear charts
            clearCharts();

            // Load available logs for offline
mode
            loadOfflineLogList();
        }
    }

    // Clear all charts
```

```
    function clearCharts() {
        photocellChart.data.labels = [];
        photocellChart.data.datasets[0].data =
[];
        photocellChart.update('none');  //
Update without animation

        masterChart.data.labels = [];
        masterChart.data.datasets[0].data = [];
        masterChart.update('none');

        smoothedValues = [];
    }

    // Update charts with current live data
    function updateCurrentData() {
        fetch('/api/current_data')
            .then(response => {
                if (!response.ok) throw new
Error('Server not available');
                return response.json();
            })
            .then(data => {
                isServerAvailable = true;

document.getElementById('connectionStatus')
.className = 'online-indicator';

document.getElementById('connectionStatus')
.innerHTML = 'Connected - Receiving Live
Data';

                // Update status

document.getElementById('sessionStart').text
Content =
                    data.session_start || 'No
session';

document.getElementById('currentMaster').te
xtContent =
                    data.current_master || 'None';

document.getElementById('knownEsps').textC
ontent =
                    data.known_esps.join(', ') ||
'None';

                // Update photocell chart with
smoothing
                const times =
data.photocell_data.map(d =>
                    new
Date(d.timestamp).toLocaleTimeString());
                const rawValues =
data.photocell_data.map(d => d.value);

                // Apply moving average
smoothing
                const smoothedValues =
movingAverage(rawValues, 5);  // 5-point
moving average

                photocellChart.data.labels =
times;

photocellChart.data.datasets[0].data =
smoothedValues;
                photocellChart.update('active');  //
Use active animation mode

                // Update master duration chart
                const masters =
Object.keys(data.master_durations);
                const durations =
Object.values(data.master_durations);
                masterChart.data.labels = masters;
                masterChart.data.datasets[0].data
= durations;
                masterChart.update('active');
            })
            .catch(error => {
                isServerAvailable = false;

document.getElementById('connectionStatus')
.className = 'offline-indicator';

document.getElementById('connectionStatus')
.innerHTML = 'Disconnected - No Live Data
Available';
            });
    }

    // Load available log files for live mode
    function loadLogList() {
        fetch('/api/logs')
            .then(response => response.json())
            .then(data => {
                const select =
document.getElementById('logSelect');
                select.innerHTML = '<option
value="">Select a log file...</option>';
                data.logs.forEach(log => {
                    const option =
document.createElement('option');
                    option.value = log;
                    option.textContent = log;
                    select.appendChild(option);
                });
            })
            .catch(error => {
                console.log('Could not load logs
from server');
            });
    }

    // Load available log files for offline
mode
    function loadOfflineLogList() {
        fetch('/api/logs')
            .then(response => response.json())
            .then(data => {
                const select =
document.getElementById('offlineLogSelect');
                select.innerHTML = '<option
value="">Select a log file from Pi...</option>';
                data.logs.forEach(log => {
                    const option =
document.createElement('option');
                    option.value = log;
                    option.textContent = log;
                    select.appendChild(option);
                });
            })
            .catch(error => {
                console.log('Raspberry Pi not
available - use file upload instead');
                const select =
document.getElementById('offlineLogSelect');
                select.innerHTML = '<option
value="">Raspberry Pi not
connected</option>';
            });
    }

    // Load local file uploaded by user
    function loadLocalFile() {
        const fileInput =
document.getElementById('fileInput');
        const file = fileInput.files[0];

        if (!file) return;
        const reader = new FileReader();
        reader.onload = function(e) {
            try {
                const data =
JSON.parse(e.target.result);
                displayLogData(data);
            } catch (error) {
                alert('Error reading file: ' +
error.message);
            }
        };
        reader.readAsText(file);
    }

    // Load log from Raspberry Pi
    function loadPiLog() {
        const filename =
document.getElementById('offlineLogSelect').
value;
        if (!filename) {
            alert('Please select a log file');
            return;
        }

        fetch(`/api/log/${filename}`)
            .then(response => response.json())
            .then(data => {
                displayLogData(data);
            })
            .catch(error => {
                alert('Error loading log from Pi: '
+ error.message);
            });
    }

    // Display log data in charts
    function displayLogData(data) {
        // Show status section

document.getElementById('offlineStatus').styl
e.display = 'block';

        // Update status info

document.getElementById('offlineSessionStar
t').textContent = data.session_start || '-';

document.getElementById('offlineSessionEnd'
).textContent = data.session_end || '-';

document.getElementById('offlineTotalPoints'
).textContent = data.total_data_points || 0;

        const masters =
Object.keys(data.master_durations || {});

document.getElementById('offlineMasters').te
xtContent = masters.join(', ') || 'None';

        // Update photocell chart with
smoothing
        const times = data.data_points.map(d
=>
            new
Date(d.timestamp).toLocaleTimeString());
        const rawValues =
data.data_points.map(d => d.value);

        // Apply moving average smoothing
        const smoothedValues =
movingAverage(rawValues, 5);

        photocellChart.data.labels = times;
        photocellChart.data.datasets[0].data =
smoothedValues;
        photocellChart.update();
```

```
            // Update master chart
            const masterLabels =
Object.keys(data.master_durations || {});
            const durations =
Object.values(data.master_durations || {});

            masterChart.data.labels =
masterLabels;
            masterChart.data.datasets[0].data =
durations;
            masterChart.update();
        }

        // Download log file
        function downloadLog() {
            const filename =
document.getElementById('logSelect').value;
            if (!filename) {
                alert('Please select a log file to
download');
                return;
            }
            window.location.href =
`/logs/${filename}`;
        }

        // Initialize on page load
        window.onload = function() {
            initCharts();
            loadLogList();

            // Start in live mode and begin updates
immediately
            switchMode('live');
        };
    </script>
</body>
</html>'''
```